Date
Page

Day 21 , 2025/11/26

141. Linked list Cycle

Approach : Floyd's Cycle Detection
we use two pointers

- Slow moves 1 node at a time (slow → slow.next)
- fast moves 2 node at a time (fast → fast.next.next)

1. Start both pointers at head.
2. Move slow by 1 step.
3. Move fast by 2 steps.
4. Compare them:
   - If slow == fast → cycle exists → return true.
   - If fast == null or fast.next == null → no cycle → return false.
5. Continue until one of these conditions happens.

Code:-

```
public boolean hasCycle(ListNode head){
    ListNode slow = head;
    ListNode fast = head;
    while(fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
        if(slow == fast){
            return true;
        }
    }
    return false;
}
```

Time Complexity : O(n)
Space Complexity : O(1)

Description | Accepted | Editorial | Solutions | Submissions

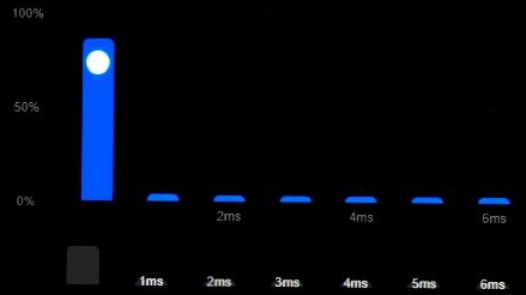
All Submissions

Runtime

0 ms | Beats **100.00%**

Memory

46.36 MB | Beats 76.40%



Code Java

Code

Java | Auto

```java
public class Solution {
    public boolean hasCycle(ListNode head) {

        ListNode slow = head;
        ListNode fast = head;
        while(fast != null && fast.next != null){
            slow = slow.next;
            fast = fast.next.next;

            if(slow == fast){
                return true;
            }
        }
        return false;
    }
}
```

Saved

Ln 22, Col 29

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =